Herald

Herald — Telegram Channel Setup Guide

Field	Value
Status	**LIVE** (HRD-011 — code complete; awaiting operator credentials for E17/E19 live evidence)
Spec ref	V3 §11.1 + §32.2
HRD	HRD-011
Env vars	`HERALD_TGRAM_BOT_TOKEN`, `HERALD_TGRAM_CHAT_ID`, `HERALD_TGRAM_LIVE_INBOUND` (optional), `HERALD_TGRAM_WEBHOOK_SECRET` (reserved for future webhook ingress)
Code	`commons_messaging/channels/tgram/` (telebot.v3 vendored at `submodules/telebot/` v3.3.8)

This guide walks you through every step needed to enable Telegram in Herald — from creating the bot to verifying the live integration test passes. Once you complete these steps, set the env vars in your shell or `.env`, hand the tokens back to the operator, and we will close HRD-011 with live E17/E19 evidence.

Table of contents

Pre-requisites

- A Telegram account (mobile, desktop, or web client). The account is YOURS; the bot you create is a separate logical entity owned by your account.

- The Herald checkout at `/Users/milosvasic/Projects/Herald` (or wherever you cloned it).
- Either:
  - `podman` or `docker` available locally (for the quickstart compose stack), OR
  - a reachable Postgres instance per the §"Postgres" entries in `OPERATOR_CREDENTIALS.md`.

Step 1 — Create a bot via @BotFather

Telegram's official "create-a-bot" interface is a bot called BotFather. You'll chat with it to create your bot.

1. Open Telegram → tap the search icon → type `BotFather`.
2. Tap the result with the verified-blue-checkmark next to its username (the real one is `@BotFather`). **Don't** tap any imposter accounts; they cannot create real bots and may phish you.
3. Start the chat with `/start`.
4. Send `/newbot` to BotFather.
5. BotFather asks for a display name. Pick something descriptive — examples:
   - `Herald Operator Bot`
   - `Herald CI Notifications`
   - `MyProject Notify Bot`
6. BotFather asks for a username. Must end in `bot` (or `_bot`). Examples:
   - `herald_operator_bot`
   - `myproject_notify_bot`
7. BotFather replies with a **bot token** of the form:

   ```
   1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t
   ```

   This token is the bot's password. Treat it like one — never paste it into a PR, a Slack channel, a commit message, or anywhere indexed.
8. **Copy the token immediately** and put it somewhere safe (password manager, secrets vault, secure note).

   Token rotation: if you ever leak this token, message `/revoke` to BotFather and pick the bot to invalidate the old token + receive a new one. This is the only fix — git history scrubbing alone does NOT invalidate a leaked Telegram token.

Step 2 — Get the chat ID

Your bot needs to know **where** to send messages. That destination is identified by a numeric **chat ID** — a chat is a one-on-one DM, a group, or a channel.

> ⚠ **Read Step 2.5 FIRST** if you intend to use a group. Group chat IDs require working around Telegram's Privacy Mode, which is **ON by default** on new bots.

Operators routinely waste time wondering why `getUpdates` returns empty until they learn this.

Choose your chat type:

Option A — Direct messages (you ⍰ bot) — works ALWAYS

This path bypasses Privacy Mode entirely. **Use it first** if Step 2.5 below hasn't yet been completed.

1. In Telegram, search for your bot's username (e.g. `@herald_operator_bot`).
2. Tap **Start** to open a DM with it. (The /start command is what introduces YOU as a chat the bot knows about.)
3. Send any message (e.g. `hello`).
4. Run the diagnostic script (recommended):

   ```
   bash scripts/tgram_diagnose.sh
   ```

   Or open in a browser:

   ```
   https://api.telegram.org/bot<YOUR-BOT-TOKEN>/getUpdates
   ```

5. The output shows `chat_id=123456789 type=private name=YourName`. The `id` value (positive integer for DMs) is your chat ID.

Option B — Group chat (bot in a group) — see Step 2.5 first

1. Create a Telegram group (or use an existing one).
2. Add your bot to the group via the group's **Add Member** menu.
3. **Complete Step 2.5 below** to ensure the bot can actually see group messages.
4. After completing Step 2.5, send a message inside the group (any text if Privacy Mode is OFF; an `@<bot-username> hi` mention OR a `/command` if Privacy Mode is ON).
5. Run `bash scripts/tgram_diagnose.sh` or visit `https://api.telegram.org/bot<TOKEN>/getUpdates`.
6. Find the `chat_id=... type=supergroup` (or `group`) block.
7. The `id` for groups is a **negative number** like `-1001234567890`.

Option C — Channel (one-way broadcast)

Channels are one-way: the bot can post but cannot read messages. Useful for notifications without two-way interaction.

1. Create a Telegram channel.
2. Add your bot as an **Administrator** (a regular member does NOT have post permission). Grant at least "Post Messages".
3. Send any message to the channel (from your own account, since the bot is admin not subscriber).
4. Run `bash scripts/tgram_diagnose.sh`.
5. Locate `chat_id=... type=channel`. Again a negative number.

Confirm the chat ID

Paste the chat ID into a temporary file or note. You'll set it as `HERALD_TGRAM_CHAT_ID` in the next step.

Step 2.5 — Privacy Mode (LOAD-BEARING for groups)

This is the #1 reason new operators get stuck. Skip it and you will waste hours wondering why `getUpdates` returns empty.

Per the official Telegram Bot API docs, when you create a bot via @BotFather, Telegram defaults its **Privacy Mode** to **ON**. With Privacy Mode ON, the bot CAN be added to a group but Telegram filters incoming group messages **before they ever reach the bot's update stream**. The bot only sees:

- `/command@bot_name` — slash commands explicitly addressed to the bot (always delivered)
- **General** `/commands` — only AFTER the bot has sent at least one message in the group (subtle: a never-spoken bot can't see bare `/commands` yet)
- **Mentions and inline replies** — messages where the bot is @-mentioned, or that reply to one of the bot's own messages
- **Service messages** — member joins/leaves, group renames, etc. (always delivered, regardless of Privacy Mode)

Plain chat messages from group members are INVISIBLE to the bot in Privacy Mode. This is why `getUpdates` is often empty even though messages clearly exist in the group.

> **Update buffer is 24h.** Per the Bot API spec, updates the bot is entitled to see are stored server-side for **up to 24 hours**. So if a message DID reach the bot's queue, you can call `getUpdates` later within that window to retrieve it.

Bot-admin shortcut (RECOMMENDED for Herald — bypasses Privacy Mode entirely)

Per the official docs: **"Bot admins always receive all messages and bypass privacy restrictions."**

If your bot is an **administrator** of the group, Privacy Mode is irrelevant. The bot sees every message. This is the simplest and most reliable setup for Herald's vertical-slice routing (Telegram → Claude Code → Telegram) because every group message routes to Claude Code, not just @-mentions.

To promote the bot to admin:

1. Open the group in Telegram.
2. Tap the group title to open group info.
3. Tap **Administrators** (or `Edit` → `Administrators`).
4. Tap **Add Admin** (or the + icon).
5. Search for your bot's username (e.g. `@atmosphere_worker_bot`) and select it.
6. Grant at minimum: **Pin Messages** + **Add New Admins** OFF, others can stay at defaults. (Telegram requires at least one permission for the admin role to be created. Even read-only admin works for our use case since we're using the admin status to bypass Privacy Mode, not to grant write powers beyond the bot's normal ones.)
7. Tap the ✓ confirmation.
8. **Send any message in the group.**
9. Run `bash scripts/tgram_diagnose.sh` — the chat ID should appear.

This works regardless of the Privacy Mode setting in @BotFather.

Check your Privacy Mode state

Run:

```
bash scripts/tgram_diagnose.sh
```

The `getMe` output includes `can_read_all_group_messages`. If it says `False`, Privacy Mode is **ON**. If `True`, it's OFF.

Two paths to fix

Path A — Quick workaround (no settings change)

In the group, send a message that EXPLICITLY contains the bot's username:

```
@atmosphere_worker_bot hi
```

(Substitute your real bot username from `getMe`.) Telegram delivers this to the bot regardless of Privacy Mode because it's a mention.

After sending, run `bash scripts/tgram_diagnose.sh` again — the chat should appear.

Common gotchas:

- Auto-correct on mobile sometimes "corrects" the bot username. Verify exactly: `@<your-bot-username>` — must match `getMe`'s `username` field verbatim.
- Slash command `/start@<bot-username>` also works (it's both a command and a mention).
- A reply to one of the bot's previous messages works too.

Path B — Disable Privacy Mode in @BotFather (best for production use)

If you want Herald to **route ALL group messages** to Claude Code (not just @-mentions), disable Privacy Mode:

1. Open Telegram → chat with `@BotFather`
2. Send `/mybots`
3. Select your bot (e.g. `@atmosphere_worker_bot`)
4. Tap **Bot Settings**
5. Tap **Group Privacy**
6. Tap **Turn off**
7. BotFather replies: "Done. The bot will see all messages."
8. **Important — the change takes effect only for new sessions**: REMOVE the bot from the group, then RE-ADD it.
9. Send any plain message in the group.
10. Run `bash scripts/tgram_diagnose.sh` — the chat should appear.

When neither path works

If you've completed Path A or B and `getUpdates` is still empty:

- **Verify the bot is actually in the group**: open the group's member list. If the bot is missing, re-add it.

- **Verify you sent the message AFTER the privacy change** (Path B): privacy changes don't retroactively re-deliver historic messages.
- **Verify you sent the message AFTER adding the bot** (always): updates from before the bot joined are never delivered.
- **Try a DM `/start` first** (Option A above): DMs ALWAYS work. If the DM chat_id appears but the group chat_id doesn't, the issue is isolated to the group's privacy configuration.
- **Check if a previous `getUpdates` consumed the update**: Telegram acks updates after each poll. Send a NEW message to force a fresh update into the queue. The diagnostic script uses `offset=-1` (read latest without acking) to mitigate this, but a previously-acked update is gone.

Diagnostic command (canonical helper)

The `scripts/tgram_diagnose.sh` helper runs all three diagnostic API calls — `getMe`, `getWebhookInfo`, `getUpdates` — and reports findings in operator-actionable form:

```
bash scripts/tgram_diagnose.sh
```

Output shape:

```
=== 1. getMe — token validity + Privacy Mode setting ===
  bot username:                @your_bot
  bot id:                      1234567890
  can_read_all_group_messages: False
  ! Privacy Mode is ON.
    - commands (/...)
    - @-mentions of itself
    - replies to its own messages
    Plain chat messages from group members are NOT delivered.

=== 2. getWebhookInfo — confirm getUpdates is the live channel ===
  no webhook configured (good)

=== 3. getUpdates — list of chats the bot has received updates from ===
  updates received: 1
  chat_id=-1001234567890  type=supergroup  name=Herald Notifications  sour
```

If `updates received: 0` with explanation, follow the suggested fix.

Step 3 — Provide the credentials to Herald

Choose ONE of these two paths (or both — the resolution order in `OPERATOR_CREDENTIALS.md` handles overlap correctly).

Path A — Shell-export (recommended for personal workstation)

Add to your `~/.zshrc` (macOS default) or `~/.bashrc` (Linux default):

```
# Herald — Telegram (HRD-011)
export HERALD_TGRAM_BOT_TOKEN='1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3o
export HERALD_TGRAM_CHAT_ID='-1001234567890'
        # or your DM chat ID
```

Then either run `source ~/.zshrc` (or `~/.bashrc`) or open a new terminal.

Verify:

```
echo "${HERALD_TGRAM_BOT_TOKEN:0:10}..."   # should print the
        first 10 chars
echo "${HERALD_TGRAM_CHAT_ID}"             # should print the
        chat ID
```

Path B — Project-local `.env`

Edit (or create) `/Users/milosvasic/Projects/Herald/.env`:

```
HERALD_TGRAM_BOT_TOKEN=1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t
HERALD_TGRAM_CHAT_ID=-1001234567890
```

Set restrictive permissions:

```
chmod 600 .env
```

For native pherald (NOT through compose), source it into your shell first:

```
set -a; source .env; set +a
```

For compose runs, `docker-compose` reads `.env` automatically.

Per §11.4.10: never `git add .env`. The repo's `.gitignore` already covers it (line 28), but always double-check before committing.

Step 4 — Verify HealthCheck

The HealthCheck integration test issues a real `getMe` call against the Bot API. It proves the token is valid and the bot is enabled.

```
cd /Users/milosvasic/Projects/Herald
go test ./commons_messaging/channels/tgram/ -tags=integration
        -run TestHealthCheck_LiveBotAPI -count=1 -timeout=60s
```

Expected:

```
--- PASS: TestHealthCheck_LiveBotAPI (1.x s)
PASS
```

If you see `SKIP`, the env vars aren't visible to `go test`. Re-source your shell or run the command with the env vars inline:

```
HERALD_TGRAM_BOT_TOKEN='...' HERALD_TGRAM_CHAT_ID='...' \
go test ./commons_messaging/channels/tgram/ -tags=integration
        -run TestHealthCheck_LiveBotAPI -count=1 -timeout=60s
```

If you see `FAIL` with `getMe returned empty body` or HTTP errors, the token is invalid. Re-check Step 1.

Step 5 — Verify Send (E17)

The E17 evidence test sends a real Telegram message to your configured chat AND persists a row in the `outbound_delivery_evidence` table. It proves the full Send + persistence round-trip works.

Pre-requisite: container runtime (podman or docker) available. The test boots a Postgres container automatically.

```
cd /Users/milosvasic/Projects/Herald
go test ./commons_messaging/channels/tgram/ -tags=integration
        -run TestSend_PersistsDeliveryEvidence -count=1
        -timeout=300s -v
```

Expected:

```
--- PASS: TestSend_PersistsDeliveryEvidence (Xs)
PASS
```

You should also see the test message arrive in your Telegram chat — something like:

```
    Herald E17 persist test 2026-05-21T11:23:45.789Z
```

The §107 anti-bluff guard asserts the persisted `channel_message_id` equals the Telegram-side `MessageID` exactly — not a Herald-generated UUID. A spoofed adapter that returned a fake ID would fail this test.

Step 6 — (Optional) Verify Subscribe + Vertical Slice (E19)

E19 is the canonical Telegram → Claude Code → Telegram round-trip. It requires:

1. All of Step 3's env vars set.
2. `HERALD_CLAUDE_BIN` resolves to a working `claude` CLI (default: looked up via `$PATH`).
3. `HERALD_CLAUDE_PROJECT_NAME` set (see ../dispatchers/CLAUDE_CODE.md).
4. `HERALD_TGRAM_LIVE_INBOUND=1` to enable the live-inbound code path.
5. **You hand-send a Telegram message to the bot during the test window** (60s default).

```
HERALD_TGRAM_LIVE_INBOUND=1 \
go test ./commons_messaging/ -tags=integration -run
        TestVerticalSlice_TelegramClaudeRoundTrip -count=1
        -timeout=300s -v
```

Within the 60s window: open Telegram → send any message to your bot. The handler will (a) receive your message, (b) dispatch it to Claude Code, (c) parse the reply, (d) send the reply back to your chat.

Expected:

```
E19 PASS: tenant=...; inbound=1; dispatch ok; outbound ok (channel_id=...)
--- PASS: TestVerticalSlice_TelegramClaudeRoundTrip (Xs)
```

You should see TWO messages in your Telegram chat: yours, then Claude's reply.

Step 7 — (Future) Webhook ingress for production

Long-poll (Steps 1-6) is what Herald uses today. For lower latency in production, Telegram supports webhook ingress — Telegram POSTs each update to your endpoint.

Status: NOT YET IMPLEMENTED — reserved for HRD-NNN.

When webhook ingress lands, you'll:

1. Generate a 256-bit hex secret:

   ```
   openssl rand -hex 32
   ```

2. Set `HERALD_TGRAM_WEBHOOK_SECRET=<the-hex>`.
3. Configure the bot's webhook URL via the Bot API:

   ```
   https://api.telegram.org/bot<TOKEN>/setWebhook?url=https://your-herald
   ```

4. Telegram POSTs each update to `/v1/webhooks/tgram` with the secret in the `X-Telegram-Bot-Api-Secret-Token` header for verification.

Until webhook ingress lands, leave `HERALD_TGRAM_WEBHOOK_SECRET` unset and rely on long-poll.

Troubleshooting

"401 Unauthorized" from `getMe`: Token is wrong. Re-copy from BotFather; check no trailing whitespace.

`getUpdates` **returns** `409 Conflict`: A webhook is configured for this bot. Either use the webhook path (Step 7) OR delete the webhook to switch back to long-poll:

```
https://api.telegram.org/bot<TOKEN>/deleteWebhook
```

Bot doesn't see group messages: For groups, you must DISABLE "Privacy Mode" in BotFather:

1. Chat with `@BotFather` → `/mybots` → select your bot → "Bot Settings" → "Group Privacy" → "Turn off".
2. Re-add the bot to the group if it was already added (the privacy change takes effect for new sessions).

Test SKIPs despite env vars set: Some test runners scope env vars per-process. Run with inline env: `HERALD_TGRAM_BOT_TOKEN='...' go test ...`.

MarkdownV2 parse error: Herald uses MarkdownV2 parse mode by default (spec §11.1). Telegram requires specific escaping for `_`, `*`, `[`, `]`, `(`, `)`, `~`, `` ` ``, `>`, `#`, `+`, `-`, `=`, `|`, `{`, `}`, `.`, `!`. If a message contains unescaped versions, the Send returns 400. Use `commons.Body.Plain` for plain-text without parse mode interpretation.

"chat not found": The bot was removed from the group, or you're using the wrong chat ID. Re-run `getUpdates` to confirm the chat exists.

Spec + code references

- **Spec**: V3 §11.1 (Telegram capabilities + Bot API surface), §32.2 (subscriber-reply long-poll loop 25s + 30s safety-net)
- **HRD**: HRD-011 (Telegram channel adapter live integration)
- **Code**:
  - `commons_messaging/channels/tgram/tgram.go` — Adapter struct + URL parser + Capabilities
  - `commons_messaging/channels/tgram/healthcheck.go` — `getMe` HealthCheck
  - `commons_messaging/channels/tgram/send.go` — Bot.Send (sendMessage MarkdownV2 + forum-topic ThreadID)
  - `commons_messaging/channels/tgram/subscribe.go` — LongPoller 25s + 30s safety-net per §32.2
  - `commons_messaging/channels/tgram/persist.go` — `NewWithStorage` + `SendForTenant` (outbound_delivery_evidence persistence)
- **Tests**:
  - `commons_messaging/channels/tgram/healthcheck_integration_test.go` — E17 prerequisite
  - `commons_messaging/channels/tgram/send_integration_test.go` — E17 (live Telegram + non-empty Receipt.ChannelMsgID)
  - `commons_messaging/channels/tgram/subscribe_integration_test.go` — E19 inbound half
  - `commons_messaging/channels/tgram/persist_integration_test.go` — E17 persistence (exact channel_message_id match)
  - `commons_messaging/vertical_slice_integration_test.go` — E19 full slice
- **Vendored SDK**: `submodules/telebot/` (gopkg.in/telebot.v3 at v3.3.8, pinned to keep `gopkg.in/telebot.v3` import-path stable per §11.4.74)
- **Related**: OPERATOR_CREDENTIALS.md §"Telegram"; ../dispatchers/CLAUDE_CODE.md for the LLM dispatch half of the vertical slice

---

Sources verified

Per HelixConstitution §11.4.99 + Herald §108.n (Latest-Source Documentation Cross-Reference Mandate). Every operator-facing instruction in this document was cross-referenced against the LATEST official online documentation of the relevant service before publication.

Last verified: 2026-05-28

Source	URL / path	Authored / verified
Telegram official Bot API documentation	https://core.telegram.org/bots/api	Step 1 (`/newbot` flow + display-name vs username vs `<bot-id>:<api-token>` token format + `/revoke` semantics); Step 2 (chat-id discovery via `getUpdates`; per-chat-type id shapes — DM positive, group negative, supergroup `-100…`, channel `-100…`); Step 4 (`getMe` token-validity probe); Step 7 (`setWebhook` /

Source	URL / path	Authored / verified
		`deleteWebhook` / `secret_token` validation header `X-Telegram-Bot-Api-Secret-Token`); Troubleshooting §401 / 409-Conflict / "chat not found"; MarkdownV2 reserved-character set.
Telegram Bot Features — Privacy Mode	https://core.telegram.org/bots/features#privacy-mode	Step 2.5 entire section (privacy-mode-ON default; what the bot DOES vs DOES NOT see; admin-bypasses-privacy invariant; the four classes of always-delivered messages — commands / mentions / replies / service messages; the 24h update buffer); the `can_read_all_group_messages` `getMe` flag semantics.
telebot.v3 vendored library	`submodules/telebot/` (v3.3.8, pinned per §11.4.74)	Step 4/5/6 integration-test mechanics (`telebot.NewBot` → `bot.Me` populated via synchronous `getMe`; `bot.Send` → `sendMessage`; `LongPoller{Timeout: 25 * time.Second}` per spec §32.2).
Empirical Herald operator testing 2026-05-28	`docs/qa/HRD-LIVE-20260528T082128Z/`	Status header (LIVE — HRD-011 code complete; E17 PASS; E19 awaiting operator-supplied live evidence); Step 5 expected output shape.
HelixConstitution §11.4.99 (this document's authority)	`<parent>/constitution/Constitution.md` §11.4.99 (HelixConstitution commit `c640947`)	This footer pattern + 90-day cadence.

Note on scope. This guide documents the Bot API path ONLY (BotFather bot tokens — long-poll + future webhook). The MTProto user-account path (operator-driven QA harness, `qaherald mtproto login` bootstrap) is documented separately at `docs/requirements/blockers/missing_env_variables.md` and `docs/guides/OPERATOR_CREDENTIALS.md` §"MTProto", which carry their own §11.4.99 Sources-verified footers covering the MTProto-specific safety rules (the `recover@telegram.org` pre-login email; no VoIP / Google Voice / Twilio / TextNow numbers; one phone = one `api_id` forever; Short-name STRICTLY alphanumeric — underscores REJECTED; ratelimit + floodwait middlewares from `submodules/gotd-td/contrib`). If you are setting up a userbot, read those documents first.

Re-verification cadence (per §11.4.99 (C)): Telegram is a risk-classified service (per §11.4.99 (D)) — Bot API breaking changes, MarkdownV2 escape-character rule changes, and webhook-secret-token validation changes all affect this guide. **90-day max staleness**, next re-verification due **2026-08-26**. Re-verify earlier on: Bot API changelog entry at https://core.telegram.org/bots/api#recent-changes, operator-error reports.